In [ ]:
```
!jupyter nbconvert --to html d3_CNN_w_Sequential_FunctionAPI.ipynb
```

In [1]:
```
# Needed libraries
import os
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import cifar10
from datetime import datetime
import time
# physical_devices = tf.config.list_physical_devices('GPU')
# if physical_devices:
#     tf.config.experimental.set_memory_growth(physical_devices[0], True)
#     print("Memory growth set for GPU.")
# else:
#     print("No GPU found. Running on CPU.")
```

In [2]:
```
(x_train, y_train),( x_test, y_test) = cifar10.load_data()
```

In [3]:
```
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

In [5]:
```
model = keras.Sequential(
    [
        keras.Input(shape=(32,32,3)),
        layers.Conv2D(32,3,padding = 'valid',activation='relu'), #1. Conv2D
        layers.MaxPooling2D(pool_size=(2,2)),
        layers.Conv2D(64,3,activation='relu'), #2. Conv2D
        layers.MaxPooling2D(),
        layers.Conv2D(128,3,activation='relu'), #3. Conv2D
        layers.Flatten(),
        layers.Dense(64,activation='relu'),
        layers.Dense(10)
    ]
)
print(model.summary())

model.compile(
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    optimizer=keras.optimizers.Adam(learning_rate=3e-4),
    metrics=["accuracy"],
)

model.fit(x_train, y_train, batch_size=64, epochs=10, verbose=2)
model.evaluate(x_test, y_test, batch_size=64, verbose=2)
```

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d_3 (Conv2D)           (None, 30, 30, 32)        896

 max_pooling2d_2 (MaxPoolin  (None, 15, 15, 32)        0
 g2D)

 conv2d_4 (Conv2D)           (None, 13, 13, 64)        18496

 max_pooling2d_3 (MaxPoolin  (None, 6, 6, 64)          0
 g2D)

 conv2d_5 (Conv2D)           (None, 4, 4, 128)         73856

 flatten_1 (Flatten)         (None, 2048)              0

 dense_2 (Dense)             (None, 64)                131136

 dense_3 (Dense)             (None, 10)                650

=================================================================
Total params: 225034 (879.04 KB)
Trainable params: 225034 (879.04 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
None
Epoch 1/10
782/782 - 31s - loss: 1.6994 - accuracy: 0.3744 - 31s/epoch - 39ms/step
Epoch 2/10
782/782 - 33s - loss: 1.3784 - accuracy: 0.5036 - 33s/epoch - 42ms/step
Epoch 3/10
782/782 - 30s - loss: 1.2432 - accuracy: 0.5603 - 30s/epoch - 39ms/step
Epoch 4/10
782/782 - 38s - loss: 1.1446 - accuracy: 0.5960 - 38s/epoch - 48ms/step
Epoch 5/10
782/782 - 44s - loss: 1.0550 - accuracy: 0.6295 - 44s/epoch - 57ms/step
Epoch 6/10
782/782 - 41s - loss: 0.9929 - accuracy: 0.6532 - 41s/epoch - 52ms/step
Epoch 7/10
782/782 - 30s - loss: 0.9403 - accuracy: 0.6743 - 30s/epoch - 38ms/step
Epoch 8/10
782/782 - 30s - loss: 0.8926 - accuracy: 0.6919 - 30s/epoch - 38ms/step
Epoch 9/10
782/782 - 29s - loss: 0.8528 - accuracy: 0.7032 - 29s/epoch - 37ms/step
Epoch 10/10
782/782 - 29s - loss: 0.8162 - accuracy: 0.7186 - 29s/epoch - 37ms/step
157/157 - 2s - loss: 0.8603 - accuracy: 0.7053 - 2s/epoch - 12ms/step
```

Out[5]: [0.8603165745735168, 0.705299973487854]

In [4]:
```python
#same model according to Function base
def my_model():
    inputs = keras.Input(shape=(32,32,3))
    x = layers.Conv2D(32,3)(inputs)
```

```
    x = layers.BatchNormalization()(x)
    x = keras.activations.relu(x)
    x = layers.MaxPooling2D()(x)
    x = layers.Conv2D(64, 5, padding='same')(x)
    x = layers.BatchNormalization()(x)
    x = keras.activations.relu(x)
    x = layers.Conv2D(128,3)(x)
    x = layers.BatchNormalization()(x)
    x = keras.activations.relu(x)
    x = layers.Flatten()(x)
    x = layers.Dense(64,activation='relu')(x)
    outputs = layers.Dense(10)(x)
    model1 = keras.Model(inputs=inputs, outputs=outputs)
    return model1

model1 = my_model()
# print(my_model())
# print(model())
print(model1.summary())
model1.compile(
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    optimizer=keras.optimizers.Adam(learning_rate=3e-4),
    metrics=["accuracy"],
)

model1.fit(x_train, y_train, batch_size=64, epochs=10, verbose=2)
model1.evaluate(x_test, y_test, batch_size=64, verbose=2)
```

```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 32, 32, 3)]       0

 conv2d (Conv2D)             (None, 30, 30, 32)        896

 batch_normalization (Batch  (None, 30, 30, 32)        128
 Normalization)

 tf.nn.relu (TFOpLambda)     (None, 30, 30, 32)        0

 max_pooling2d (MaxPooling2  (None, 15, 15, 32)        0
 D)

 conv2d_1 (Conv2D)           (None, 15, 15, 64)        51264

 batch_normalization_1 (Bat  (None, 15, 15, 64)        256
 chNormalization)

 tf.nn.relu_1 (TFOpLambda)   (None, 15, 15, 64)        0

 conv2d_2 (Conv2D)           (None, 13, 13, 128)       73856

 batch_normalization_2 (Bat  (None, 13, 13, 128)       512
 chNormalization)

 tf.nn.relu_2 (TFOpLambda)   (None, 13, 13, 128)       0

 flatten (Flatten)           (None, 21632)             0

 dense (Dense)               (None, 64)                1384512

 dense_1 (Dense)             (None, 10)                650

=================================================================
Total params: 1512074 (5.77 MB)
Trainable params: 1511626 (5.77 MB)
Non-trainable params: 448 (1.75 KB)
_________________________________________________________________
None
Epoch 1/10
782/782 - 161s - loss: 1.3257 - accuracy: 0.5271 - 161s/epoch - 207ms/step
Epoch 2/10
782/782 - 147s - loss: 0.9286 - accuracy: 0.6739 - 147s/epoch - 188ms/step
Epoch 3/10
782/782 - 150s - loss: 0.7594 - accuracy: 0.7324 - 150s/epoch - 192ms/step
Epoch 4/10
782/782 - 145s - loss: 0.6481 - accuracy: 0.7713 - 145s/epoch - 186ms/step
Epoch 5/10
782/782 - 151s - loss: 0.5511 - accuracy: 0.8059 - 151s/epoch - 193ms/step
Epoch 6/10
782/782 - 154s - loss: 0.4715 - accuracy: 0.8351 - 154s/epoch - 197ms/step
Epoch 7/10
782/782 - 145s - loss: 0.3922 - accuracy: 0.8633 - 145s/epoch - 185ms/step
```

```
Epoch 8/10
782/782 - 145s - loss: 0.3272 - accuracy: 0.8865 - 145s/epoch - 186ms/step
Epoch 9/10
782/782 - 143s - loss: 0.2700 - accuracy: 0.9072 - 143s/epoch - 183ms/step
Epoch 10/10
782/782 - 148s - loss: 0.2186 - accuracy: 0.9271 - 148s/epoch - 190ms/step
157/157 - 10s - loss: 1.1094 - accuracy: 0.6931 - 10s/epoch - 66ms/step
```

Out[4]: [1.1093515157699585, 0.6930999755859375]

In [ ]: